LAB: GPIO Digital InOut(eval board)

LAB: GPIO Digital InOut

Date: 2026-09-02

Author/Partner: Juyoung Lee

Demo Video: [EC 25-2] LAB (GPIO InOut)

PDF version:

"LAB_GPIO_InOut_22200576_JuyoungLee.pdf" could not be found.

Introduction

In this lab, you are required to create a simple program that toggles multiple LEDs using a push-button input.

You must create your own drivers for GPIO digital input and output control.

You must submit

- LAB Report (*.pdf)
- Zip source files(main.c, ecRCC.h, ecGPIO.h etc...).
  - Only the source files.
  - Do not submit project files

Requirement

Hardware

- MCU
  - NUCLEO-F411RE
- Eval Board
- Actuator/Sensor/Others:

Software

- PlatformIO, CMSIS, EC_HAL library

Problem 0: STM-Arduino

We are going to create a simple program that turns LED(LD2) on and off by pressing the user button(BT1), using Arduino Syntax

GPIO Digital In/Out

Create a new project under the directory `\repos\EC\lab\`

- The project folder name is "**\LAB_GPIO_DIO_LED".**

Configures the specified pin to behave either as an input or an output.

```
pinMode(pin, mode)
```

- pin: the pin number to set the model of.
- mode: INPUT, OUTPUT or INPUT_PULLUP.

> Look up for pinMode() function in arduino reference for detail description.

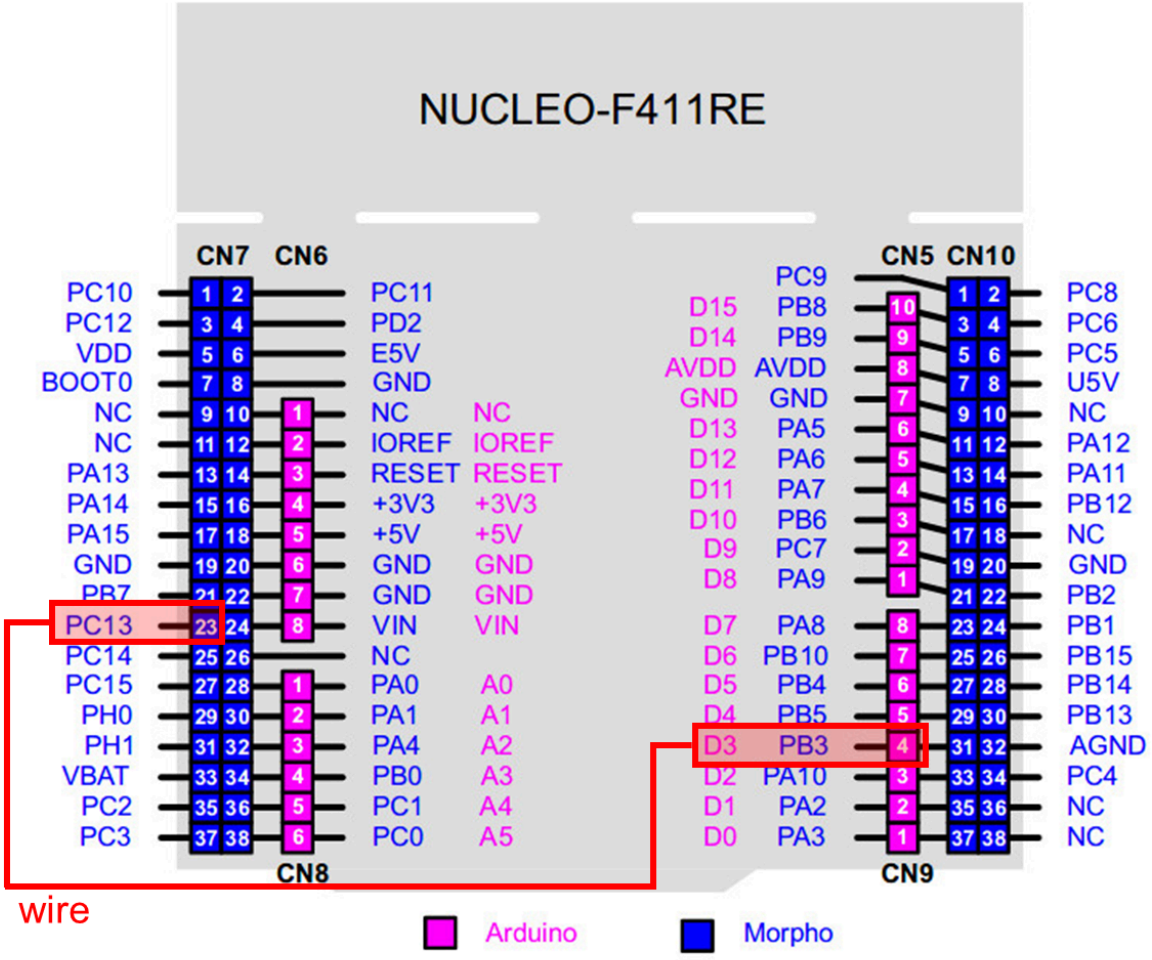
Example code

Open *Arduino IDE* and Create a new program named as '**TU_arduino_GPIO_LED_button.ino**'.

Write the following source code: source code.

```
const int btnPin = 3;
const int ledPin = 13;

int btnState = HIGH;

void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(btnPin, INPUT);
}

void loop() {
  btnState = digitalRead(btnPin);

  if (btnState == HIGH)
    digitalWrite(ledPin, LOW);

  else
    digitalWrite(ledPin, HIGH);

}
```

The user button pin is `PC13`, but this pin cannot be used in arduino. So, you should connect `PC13` to `pinName D3` by using wire.



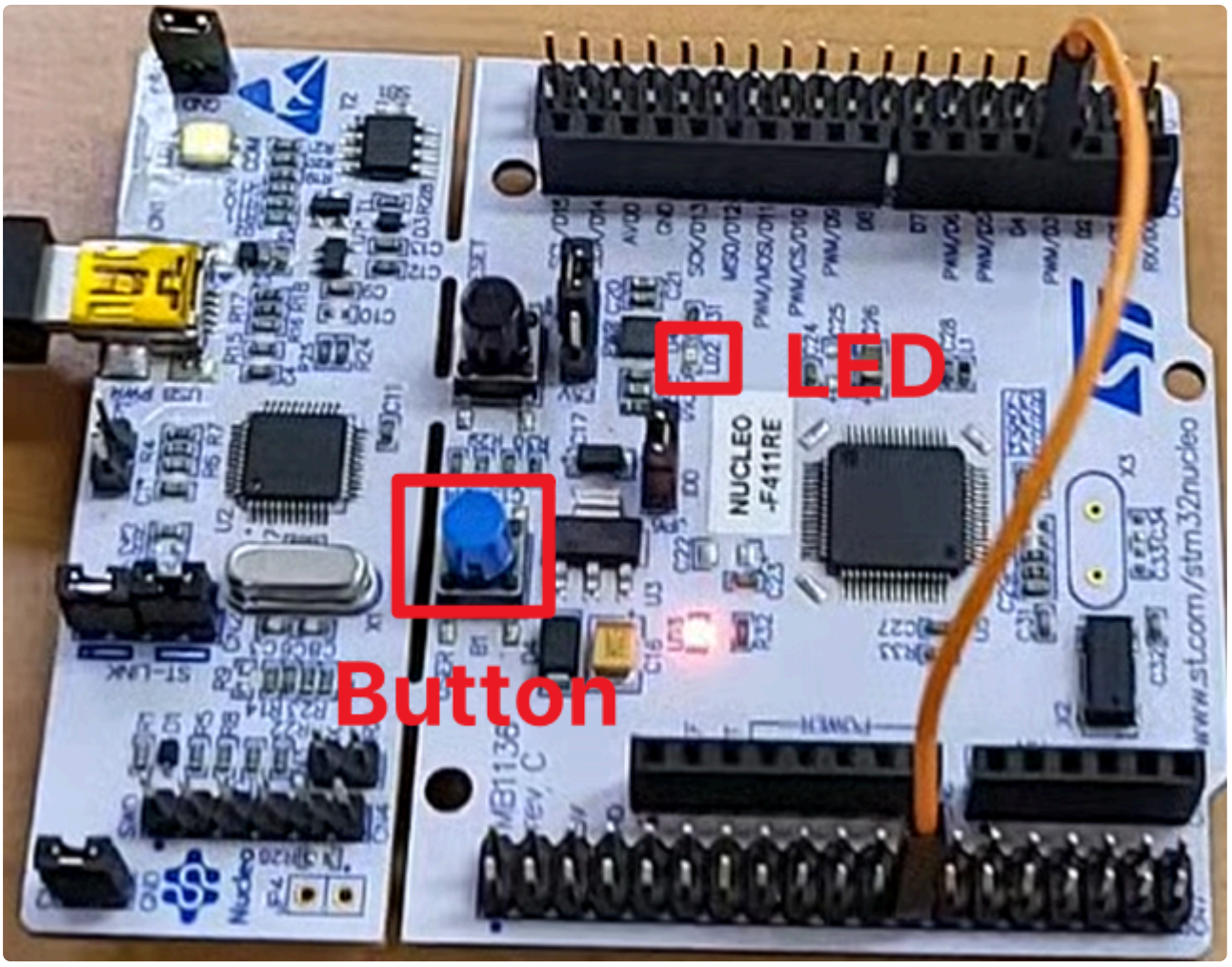
Click on **upload** button. Push the reset button(black) and check the performance.

The LED(LD2) should be turned on when the button is pressed.

FSM

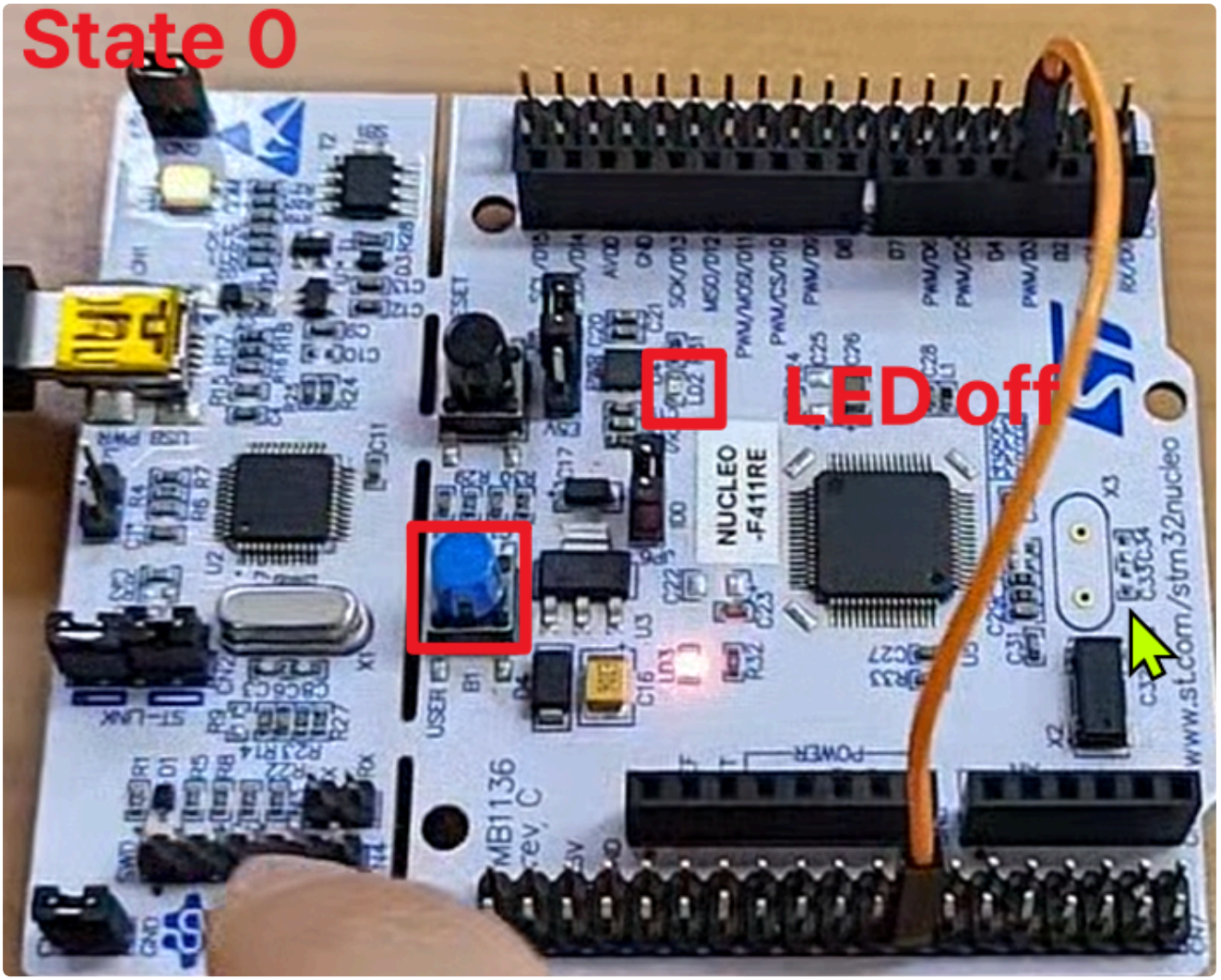
State	Next State (but = 0)	Next State (but = 1)	Output
S0	S0	S1	LED off
S1	S0	S1	LED on

Circuit

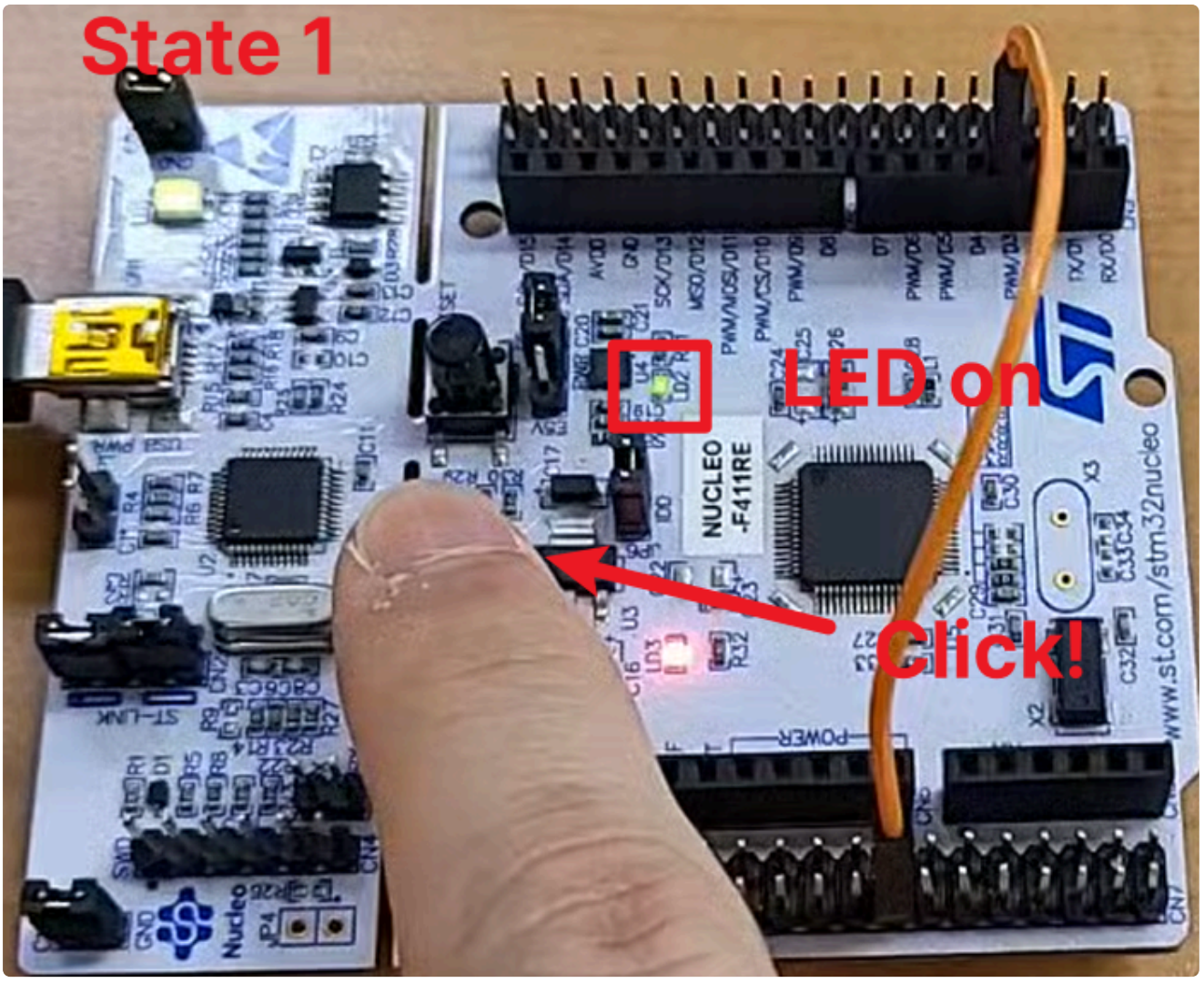


Results

Output of State 0



Output of State 1



Demo Video
[EC 25-2] LAB (GPIO InOut) - YouTube

Problem 1: Create EC_HAL library

Procedure

Prepare Library Files

Manage your library in the directory of `\repos\EC\include\`.

> DO NOT make duplicates of library files under each project folders

- **For VS.Code User:** Save your header library files in this directory. See here for detail.
- **For uVision User:** Save your header library files in this directory. See here for detail.

Download library files.

- `ecRCC2.h, ecRCC2.c`
- `ecPinNames.h, ecPinNames.c`

- `ecGPIO2_student.h, ecGPIO2_student.c`

Create your own library for GPIO by renaming the downloaded files (`ecGPIO2_student.*`) as

- `ecGPIO2.c`, `ecGPIO2.h`
- Open `ecGPIO2.c` and change from `#include "ecGPIO2_student.h"` to `#include "ecGPIO2.h"`
- Write your name and modified date in the comment section

Function Definitions

Fill-in the blanks in the header files to complete the definitions for the provided functions

ecGPIO2.h

```
void GPIO_init(PinName_t pinName, int mode);
void GPIO_write(PinName_t pinName, int Output);
int  GPIO_read(PinName_t pinName);
void GPIO_mode(PinName_t pinName, int mode);
void GPIO_ospeed(PinName_t pinName, int speed);
void GPIO_otype(PinName_t pinName, int type);
void GPIO_pupd(PinName_t pinName, int pupd);
```

GPIO_init

```
void GPIO_init(PinName_t pinName, uint32_t mode){
    GPIO_TypeDef * Port;
    unsigned int pin;
    ecPinmap(pinName, &Port, &pin);

    // mode  : Input(0), Output(1), AlterFunc(2), Analog(3)
    if      (Port == GPIOA)       RCC_GPIOA_enable();
    else if (Port == GPIOB)     RCC_GPIOB_enable();
    else if (Port == GPIOC)     RCC_GPIOC_enable();
    else if (Port == GPIOD)     RCC_GPIOD_enable();
    else if (Port == GPIOE)     RCC_GPIOE_enable();
    else if (Port == GPIOH)     RCC_GPIOH_enable();

    GPIO_mode(pinName, mode);
}
```

GPIO_write

```
// Write digital output state
void GPIO_write(PinName_t pinName, uint32_t Output){
```

```
    GPIO_TypeDef *Port;
    unsigned int pin;
    ecPinmap(pinName, &Port, &pin);

    if (Output == 0) Port->ODR &= ~(1UL << pin);
    else             Port->ODR |=  (1UL << pin);
}
```

GPIO_read

```
uint32_t GPIO_read(PinName_t pinName){
    GPIO_TypeDef * Port;
    unsigned int pin;
    ecPinmap(pinName,&Port,&pin);

    return (Port->IDR >> pin) & 1UL;
}
```

GPIO_mode

```
// Input(00), Output(01), AlterFunc(10), Analog(11, reset)
void GPIO_mode(PinName_t pinName, uint32_t mode){
    GPIO_TypeDef *port;
    unsigned int pin;
    ecPinmap(pinName, &port, &pin);

    port->MODER &= ~(3UL<<(2*pin));
    port->MODER |= mode<<(2*pin);
}
```

GPIO_ospeed;

```
// GPIO Speed : Low speed (00), Medium speed (01), Fast speed (10), High
speed (11)
void GPIO_ospeed(PinName_t pinName, uint32_t speed){
    GPIO_TypeDef * Port;
    unsigned int pin;
    ecPinmap(pinName,&Port,&pin);

    Port->OSPEEDR &=  ~(3UL <<(2*pin));
    Port->OSPEEDR |=   speed <<(2*pin);
}
```

GPIO_otype;

```
// GPIO Output Type: Output push-pull (0, reset), Output open drain (1)
void GPIO_otype(PinName_t pinName, uint32_t type){
    GPIO_TypeDef * Port;
    unsigned int pin;
    ecPinmap(pinName,&Port,&pin);

    Port->OTYPER &=  ~(1UL << pin);
    Port->OTYPER |=  type << pin;
}
```

GPIO_pupd);

```
// GPIO Push-Pull    : No pull-up, pull-down (00), Pull-up (01), Pull-down
(10), Reserved (11)
void GPIO_pupd(PinName_t pinName, uint32_t pupd){
    GPIO_TypeDef * Port;
    unsigned int pin;
    ecPinmap(pinName,&Port,&pin);

    Port->PUPDR &= ~(3UL << pin);
    Port->PUPDR |=   (pupd <<pin);
}
```

Problem 2: Toggle a single LED with Digital Sensor(Photodetector)

Problem

Toggle the LED by covering the photodetector sensor.

- Dark (LED ON), Bright (LED OFF) and repeat
- Connect the Photodetector sensor to PA_0 pin on Nucleo Board.

Procedure

1. Connect the evaluation board (JKIT-NUCLEO) to the MCU.
2. Make sure that your library **ecGPIO2.h, ecGPIO2.c** are in `EC\include\`.
3. Create a new project under the directory `EC\lab\`

- Environment: “**LAB_GPIO_DIO_LED_Photosensor”.**
- Source File: “**LAB_GPIO_DIO_LED_Photosensor.c”**

4. You must modify the `platformio.ini` , to add new environment.

Configuration

Configure the MCU GPIO

Function	Port - Pin	Configuration
Photodetector on JKIT	PA_0	Digital IN, Pull-Up
LED0	PB_12	Digital OUT, Pull-UP, Open-Drain, Medium Speed

Code

Hardware Configuration

- **Photosensor Input**: Pin `PA_0` (Digital Input)
- **LED Output**: Pin `PB_12` (Digital Output)

Code Breakdown

- `setup()` **Function**
  - `RCC_HSI_init()` : Initializes the High-Speed Internal (HSI) system clock.
  - `GPIO_init(Photosensor_PIN, INPUT)` : Configures pin `PA_0` as a digital input to read the photosensor status.
  - `GPIO_init(LED_PIN, OUTPUT)` : Configures pin `PB_12` as a digital output to drive the LED.
- `main()` **Function & Infinite Loop**
  - `GPIO_read(Photosensor_PIN)` : Reads the current digital voltage state (`HIGH` or `LOW`) from the photosensor.
  - **Control Logic**:
    - **Photosensor Active (`1`)**: Writes `LOW` to `PB_12` , turning the LED **OFF**.
    - **Photosensor Inactive (`0`)**: Writes `HIGH` to `PB_12` , turning the LED **ON**.
  - `delay_ms_HSI(100)` : Introduces a 100 ms delay between readings to ensure stable execution.

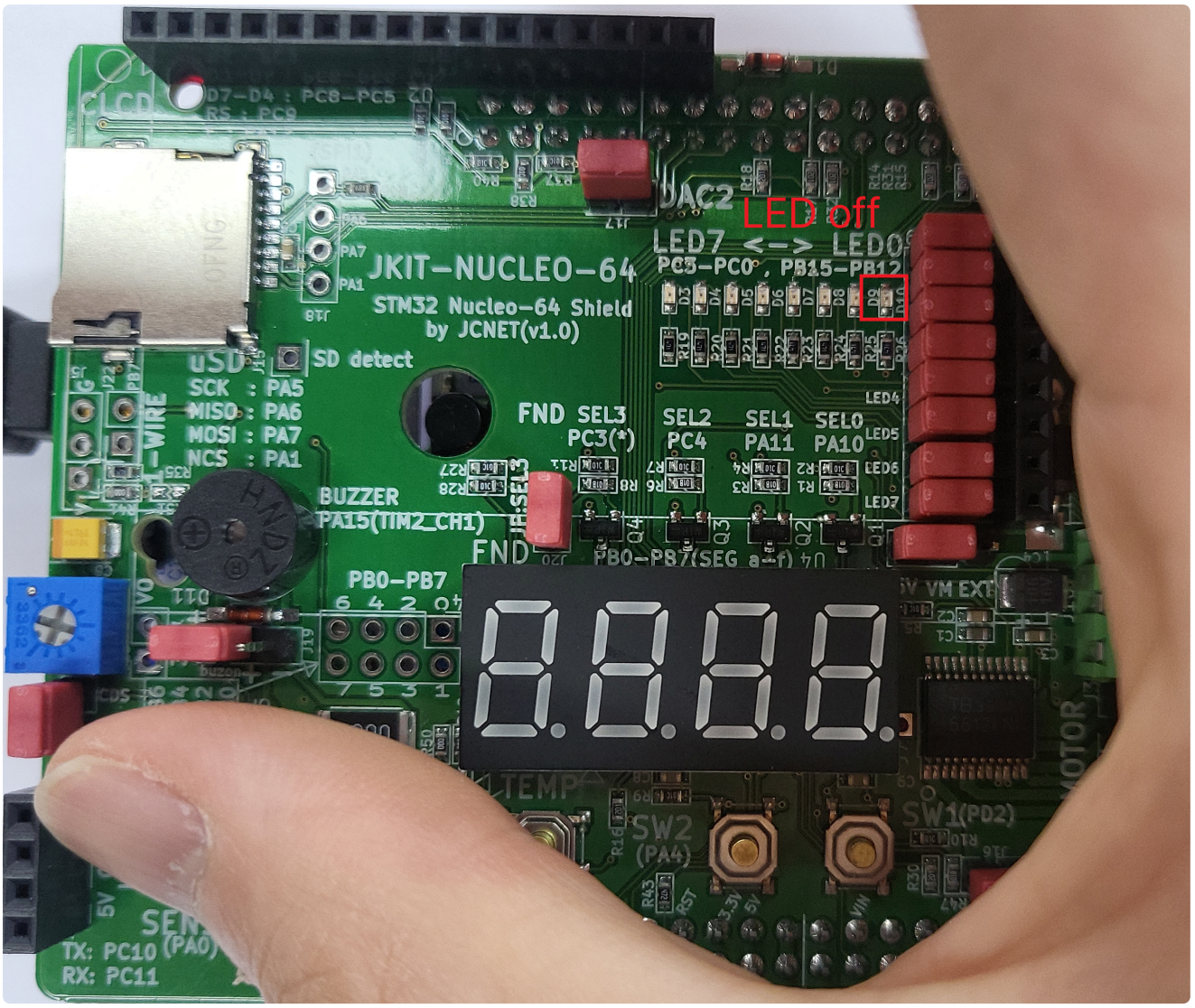
Sample Code: example code

```
/*-----------------------------------------------------------\
@ Embedded Controller by Young-Keun Kim - Handong Global University
Author           : [ Juyoung Lee ]
Created          : 09-11-2026
Modified         : 09-12-2026
Language/ver     : C in VS Code
```

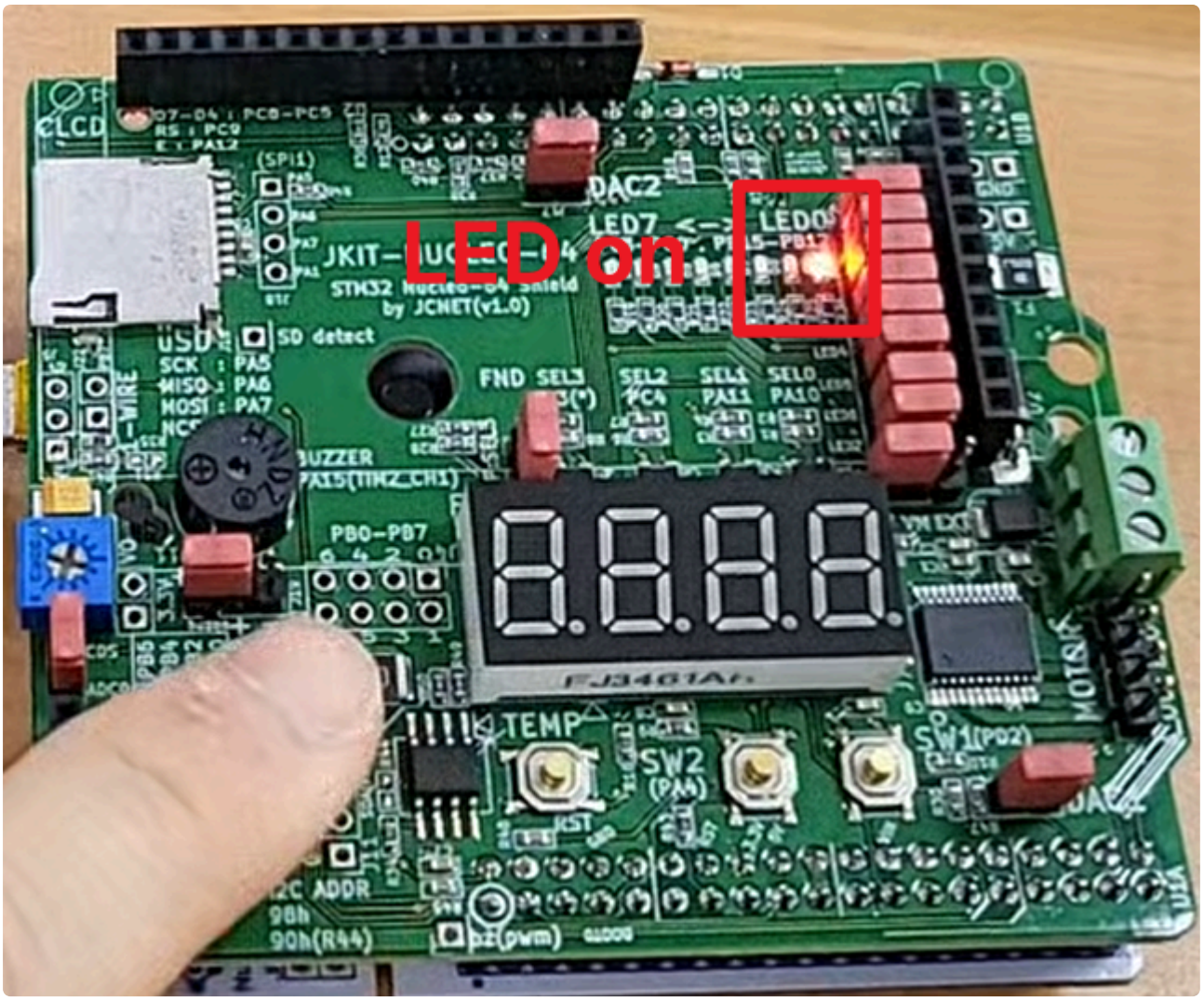
```
Description      : Control PB_12 LED state using PA_0 Photodetector digital
input
/----------------------------------------------------------------*/

#include "ecRCC2.h"
#include "ecGPIO2.h"

#define LED_PIN PB_12
#define BUTTON_PIN PA_0

// Initialiization
void setup(void) {
    RCC_HSI_init();

    // initialize the pushbutton pin as an input:
    GPIO_init(BUTTON_PIN, INPUT);
    GPIO_pupd(BUTTON_PIN, PULL_UP);

    // initialize the LED pin as an output:
    GPIO_init(LED_PIN, OUTPUT);
    GPIO_pupd(LED_PIN, PULL_UP);
    GPIO_otype(LED_PIN, OPEN_DRAIN);
    GPIO_ospeed(LED_PIN, MEDIUM);

}

int main(void) {
    setup();
    int buttonState=0;

    while(1){
        // check if the pushbutton is pressed. Turn LED on/off accordingly:
        buttonState = GPIO_read(BUTTON_PIN);
        if(buttonState)  GPIO_write(LED_PIN, LOW);
        else         GPIO_write(LED_PIN, HIGH);
        delay_ms_HSI(100);                                  // delay 100 ms
    }
}
```

Result

1) Open-Drain mode



2) Push-pull mode



Demo Video: [EC 25-2] LAB (GPIO InOut) - YouTube

- Start Time: 11 seconds (0:11)

Discussion

1. Why does the LED fail to turn on when the STM32 GPIO pin is configured in Open-Drain mode?

   > When configured in open-drain mode, the generated output voltage fails to exceed the threshold voltage of the LED, causing the LED to remain off.

Problem 3: Toggle multiple LEDs with a button

Problem

Use the four LEDs (LED0 to LED3) on the JKIT evaluation board.

Each time Button B1 is pressed, turn on one LED at a time in a sequential loop

- e.g., LED0 -> LED1 -> LED2 -> LED3 -> LED0)

Procedure

1. Connect the evaluation board (JKIT-NUCLEO) to the MCU.
2. Make sure that your library **ecGPIO2.h, ecGPIO2.c** are in `EC\include\`.
3. Create a new project under the directory `EC\lab\`

- Environment: "**LAB_GPIO_DIO_multiLED**"
- Source File: ""**LAB_GPIO_DIO_multiLED.c**"

4. You must modify the `platformio.ini` , to add new environment.

> You MUST write your name in the top of the source file, inside the comment section.

Configuration

Function	Port - Pin	Configuration
SW2 on JKIT	PA_4	Digital IN, Pull-Up
LED_0~LED_3	PB_12, PB_13, PB_14, PB_15	Digital OUT, Pull-UP, Open-Drain, Medium Speed

State table

State	Next State(but=0)	Next State(but=1)	Output
S0	S0	S1	LED0 on, Other off
S1	S1	S2	LED1 on, Other off
S2	S2	S3	LED2 on, Other off
S3	S3	S0	LED3 on, Other off

Code

setup()

1. internal clock(HSI)
2. Sets pin PA_4 as an input for the switch
3. Configures pins PB_12-PB_15 as outputs for the LEDs

main()

1. Edge Detection

- Continuously polls PA_4 to detect a falling edge (HIGH -> LOW transition when the button is pressed).

2. Debouncing

- Use a 30 ms delay (delay_ms_HSI(30)) as software debouncing to filter out mechanical contact chatter before confirming the press.

LED State Control (updateLEDs()):

- Drives the active LED pin HIGH (ON) and sets all other three pins Low(OFF)

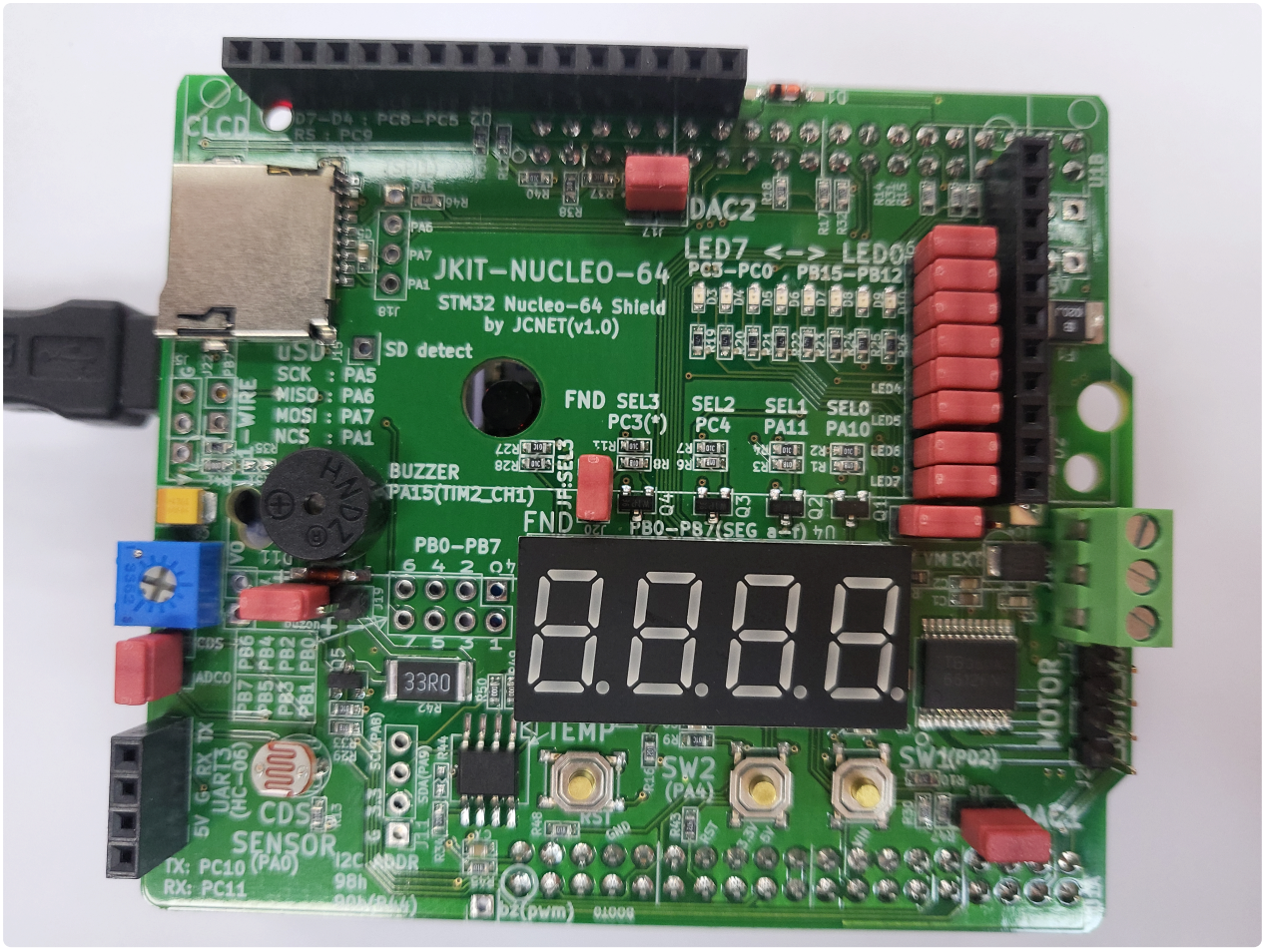
```
/*-----------------------------------------------------------------\
@ Embedded Controller by Young-Keun Kim - Handong Global University
Author           : Juyoung Lee
Created          : 09-11-2026
Modified         : 09-12-2026
Language/ver     : C in VS Code

Description      : Control PB_12, PB_13, PB_14, PB_15 LEDs sequentially
using PA_4 button input
/-----------------------------------------------------------------*/

#include "ecRCC2.h"
#include "ecGPIO2.h"

#define NUM_LED    4
#define BUTTON_PIN PA_4

#define BUTTON_PUPD     (int)(0)

#define LED_TYPE        (int)(0)
#define LED_SPEED       (int)(0)
#define LED_PUPD        (int)(0)

const PinName_t ledPins[NUM_LED] = {PB_12, PB_13, PB_14, PB_15};

void updateLEDs(int activeIndex);

// Initialization
void setup(void) {
    RCC_HSI_init();

    // 1. Initialize SW2 Button (PA_4) as Input with Pull-Up
    GPIO_init           (BUTTON_PIN, INPUT);
```

```
    GPIO_pupd           (BUTTON_PIN, BUTTON_PUPD);

    // 2. Initialize LEDs (PB_12 ~ PB_15) as Output (Open-Drain, Pull-Up,
Medium Speed)
    for(int i = 0; i < NUM_LED; i++) {
        GPIO_init       (ledPins[i], OUTPUT);
        GPIO_pupd       (ledPins[i], LED_PUPD);      // Pull-Up
        GPIO_otype      (ledPins[i], LED_TYPE);      // Open-Drain
        GPIO_ospeed     (ledPins[i], LED_SPEED);     // Medium Speed
    }

    // Turn ON LED0 initially, turn OFF others
    updateLEDs(0);
}

int main(void) {
    setup();

    int currentButtonState = HIGH;
    int prevButtonState    = HIGH;
    int ledIndex           = 0;

    while(1) {
        currentButtonState = GPIO_read(BUTTON_PIN);

        // Detect Falling Edge (Button Pressed: HIGH -> LOW)
        if (prevButtonState == HIGH && currentButtonState == LOW) {
            delay_ms_HSI(30); // Software Debouncing delay

            if (GPIO_read(BUTTON_PIN) == LOW) {
                // Increment LED index sequentially (0 -> 1 -> 2 -> 3 -> 0)
                ledIndex = (ledIndex + 1) % NUM_LED;
                updateLEDs(ledIndex);
            }
        }
        prevButtonState = currentButtonState;
    }
}

// Function to control LED states (Turn ON activeIndex LED, Turn OFF all
other LEDs)
void updateLEDs(int activeIndex) {
    for (int i = 0; i < NUM_LED; i++) {
        if (i == activeIndex)   GPIO_write(ledPins[i], HIGH);
        else                    GPIO_write(ledPins[i], LOW);

    }
}
```

Results

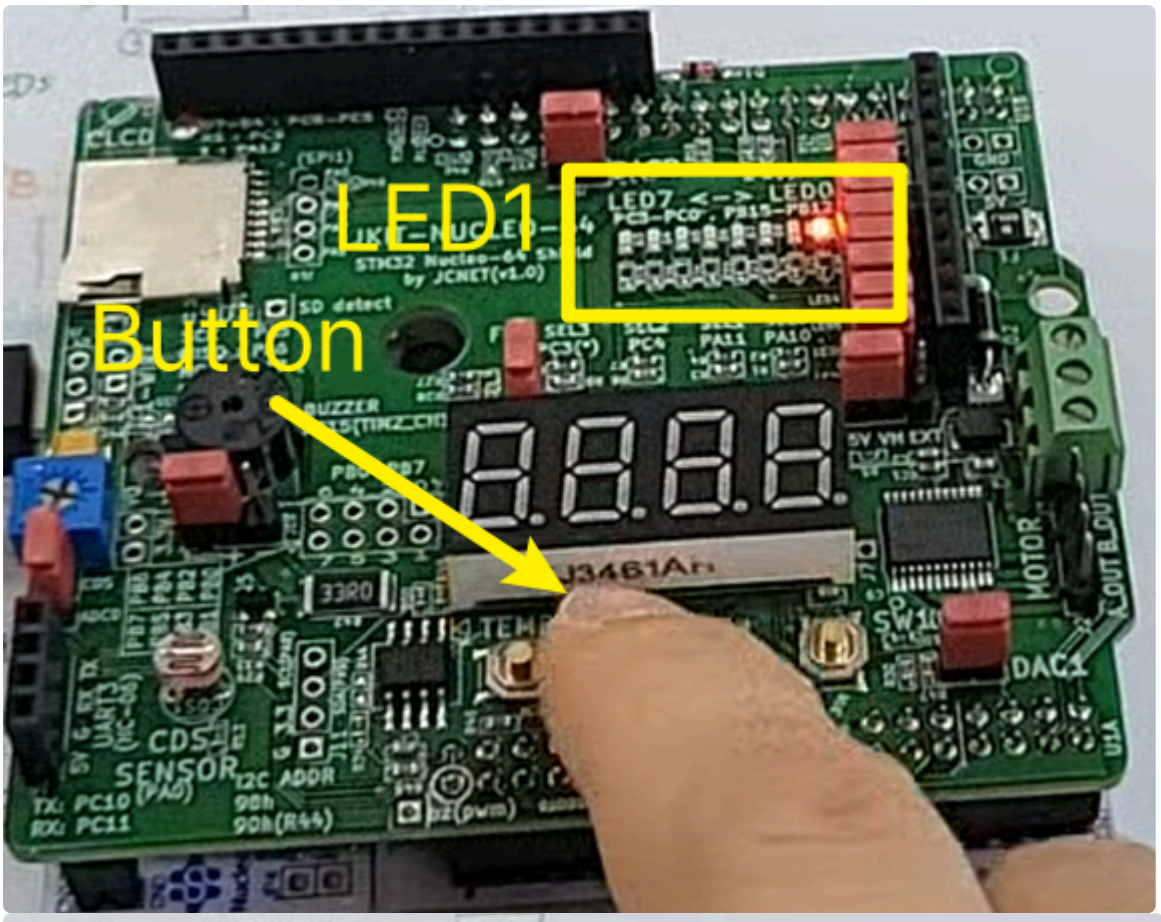
1) Open-drain mode

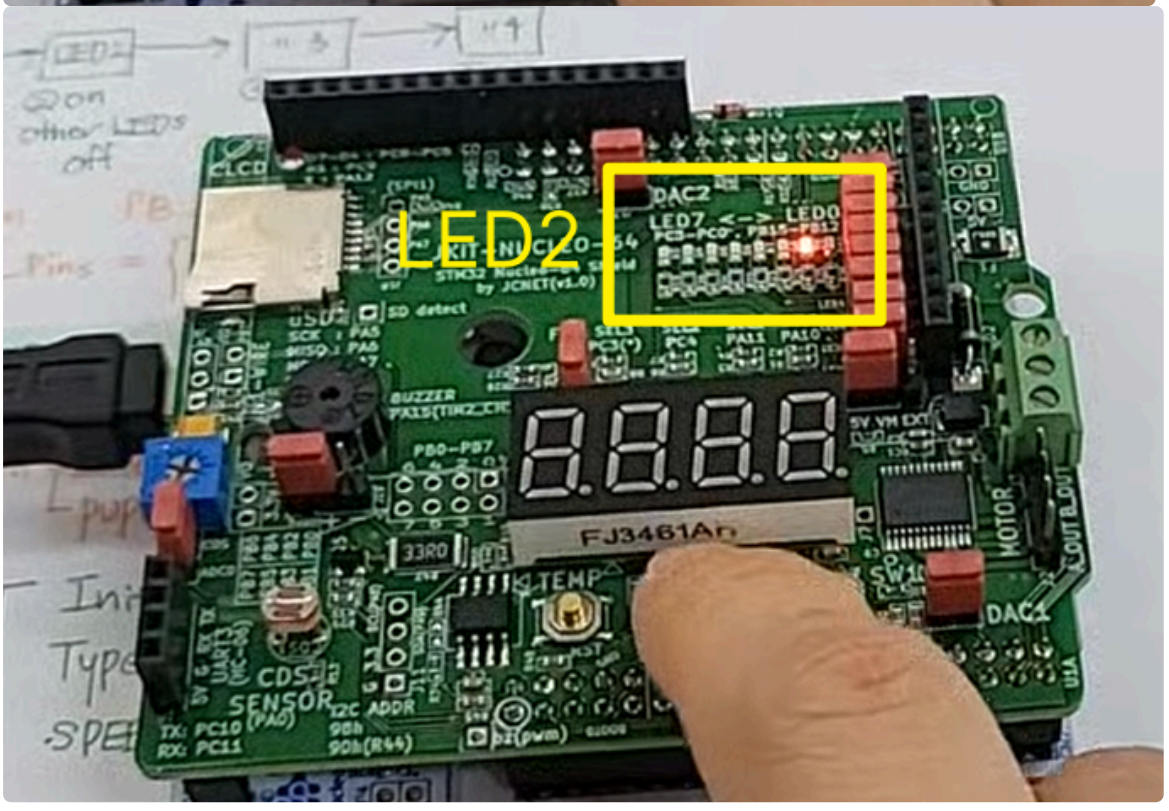
- The LED fail to turn on

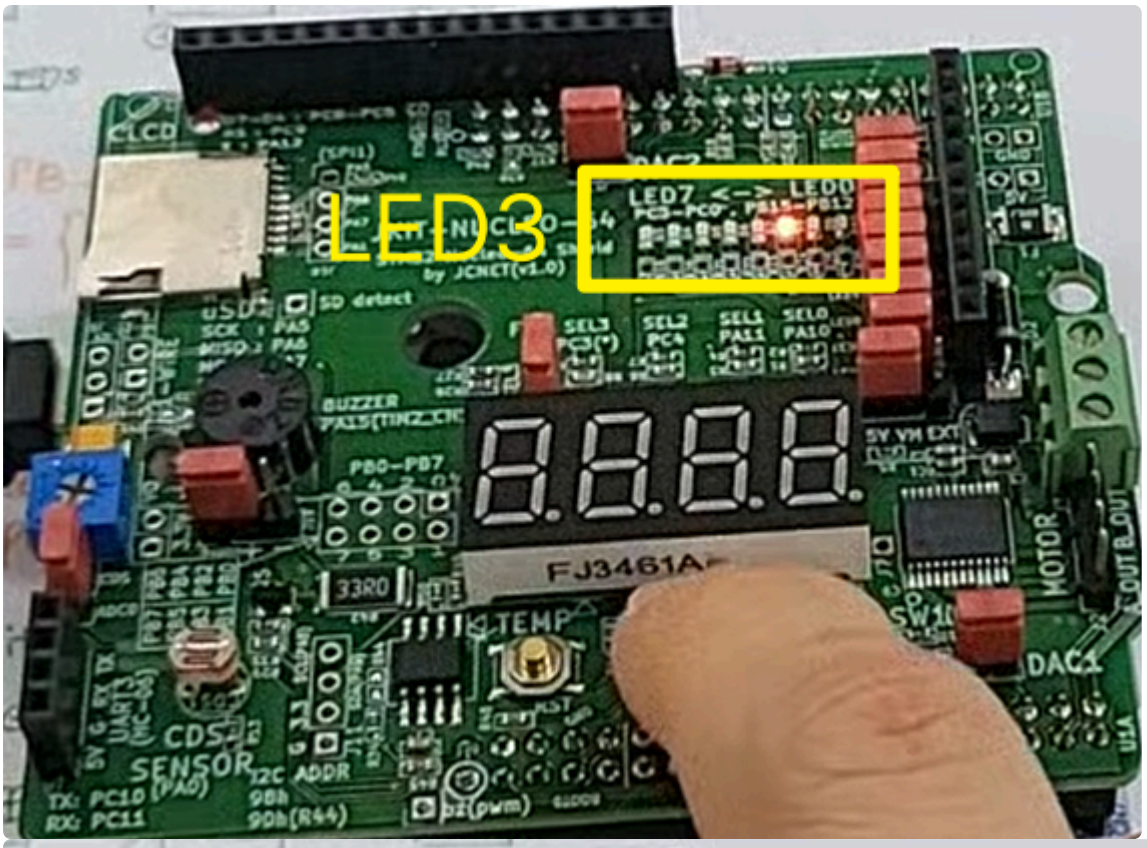


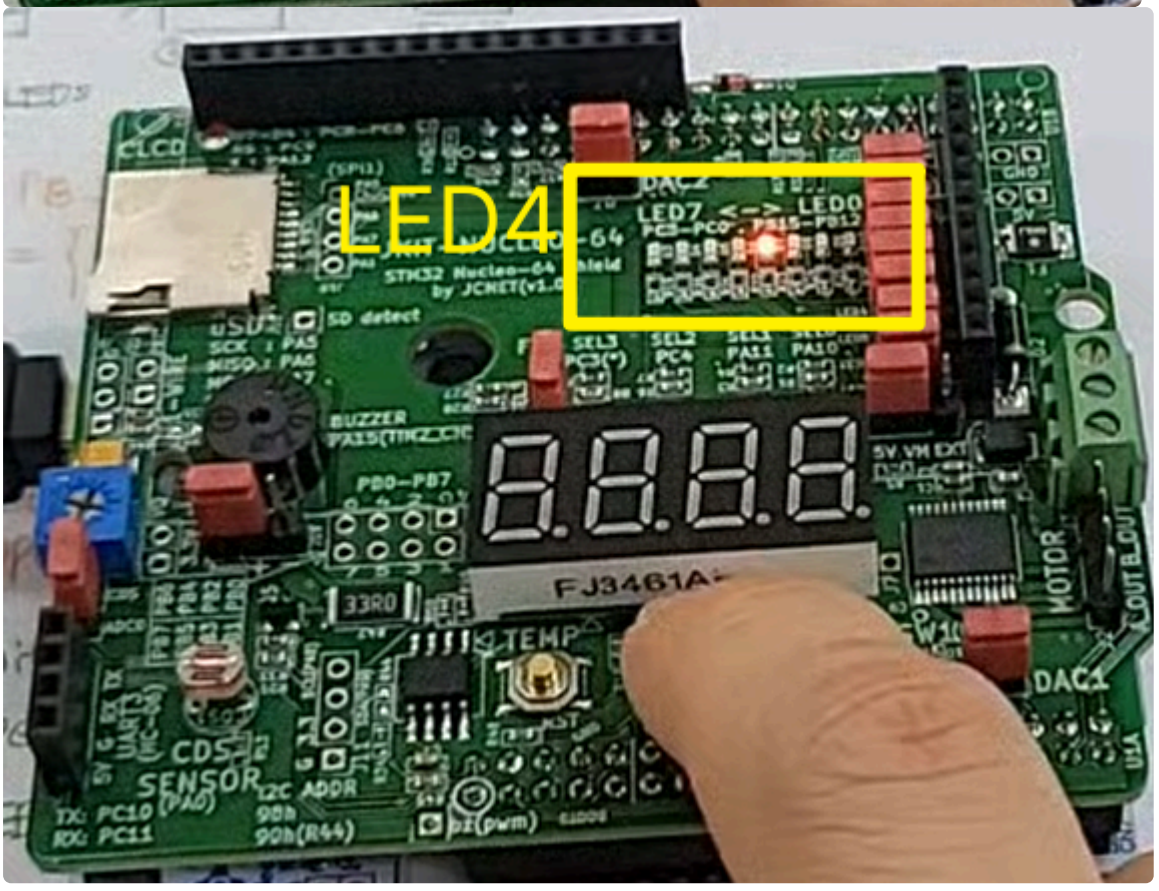
2) Push-pull mode

- A brief explanation: Each time Button B1 is pressed, turn on one LED at a time in a sequential loop (LED1 -> LED2 -> LED3 -> LED4 -> LED1)


LED1
Button


LED2





Demo Video: [EC 25-2] LAB (GPIO InOut) - YouTube

- Start Time: 35 seconds (0:35)

Discussion

1. Find out a typical solution for software debouncing and hardware debouncing.
2. What method of debouncing did this NUCLEO board use for the push-button(B1)?

3. Why does the LED fail to turn on when the STM32 GPIO pin is configured in Open-Drain mode?

Answer to Q1

> 1. Software Debouncing:
>   1. Detects an initail signal edge, delays execution for 10 to 50 ms until mechanical chatter settles, and re-reads the pin state to confirm the input
> 2. Hardware Debouncing:
>   1. Connects a LPF across the switch contacts to smooth out voltage spikes and contact chatter into a continuous voltage curve.
>   2. Use Schmitt trigger to create crisp logic transitions using voltage hysteresis

Answer to Q2

> Circuit Layout: Push-button B1 is wired in parallel with a capacitor and resistor network

Answer to Q3

> When configured in open-drain mode, the generated output voltage fails to exceed the threshold voltage of the LED, causing the LED to remain off.

Reference

Complete list of all references used (github, blog, paper, etc)


```
STMicroelectronics, [STM32F411xC/xE Datasheet: Production Data]
(https://www.st.com/resource/en/datasheet/stm32f411ce.pdf).
```


Troubleshooting

(Option) You can write a Troubleshooting section